

4.4 贪心法的典型应用

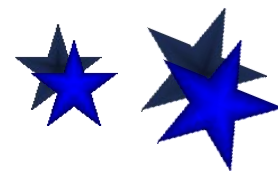
算法实现与时间复杂度

建立FIND数组， $\text{FIND}[i]$ 是结点 i 的连通分支标记。

- (1) 初始 $\text{FIND}[i] = i$.
- (2) 连通分支合并，较小分支标记更新为较大分支标记

每个结点至多更新 $\log n$ 次，
建立和更新 FIND 数组: $O(n \log n)$

时间: $O(m \log m) + O(n \log n) + O(m)$
 $= O(m \log n)$



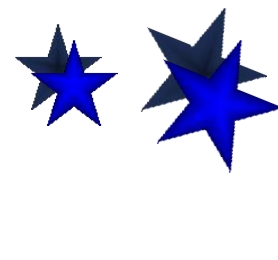
4.4 贪心法的典型应用

应用：数据分组问题

一组数据(照片, 文件, 生物标本)要把它们按照相关性进行分类.

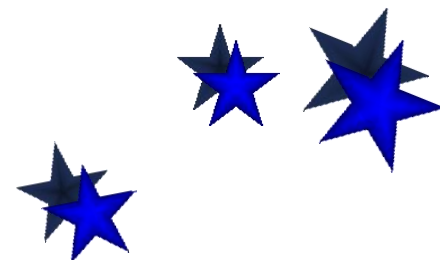
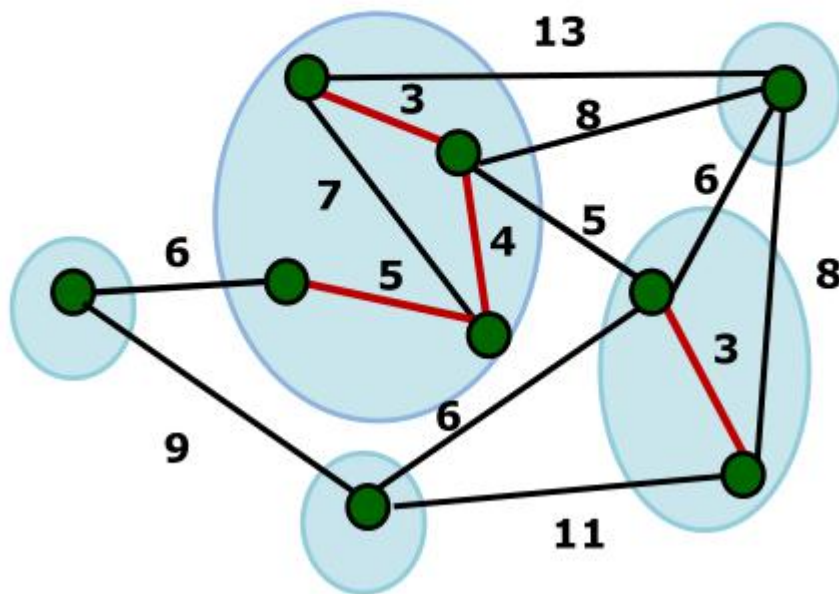
用相似度函数或“距离”来描述个体之间的差距.

如果分成5类, 使得每类内部的个体尽可能相近, 不同类之间的个体尽可能地“远离”. 如何划分?



4.4 贪心法的典型应用

应用：数据分组问题

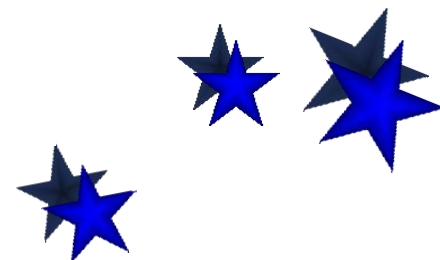


4.4 贪心法的典型应用

单链聚类

类似于Kruskal算法

- (1) 按照边长从小到大对边排序
- (2) 依次考察当前最短边 e ，如果 e 与已经选中的边不构成回路，则把 e 加入集合，否则跳过 e . 计数图的连通分支个数.
- (3) 直到保留了 k 个连通分支为止.



4.4 贪心法的典型应用

- 单源最短路径：Dijkstra算法

归纳证明思路

命题：当算法进行到第 k 步时，对于 S 中每个结点 i ,

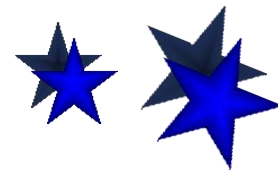
$$dist[i] = short[i]$$

归纳基础

$$k = 1, S = \{s\}, dist[s] = short[s] = 0.$$

归纳步骤

证明：假设命题对 k 为真，则对 $k+1$ 命题也为真。



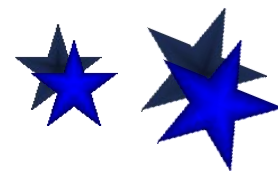
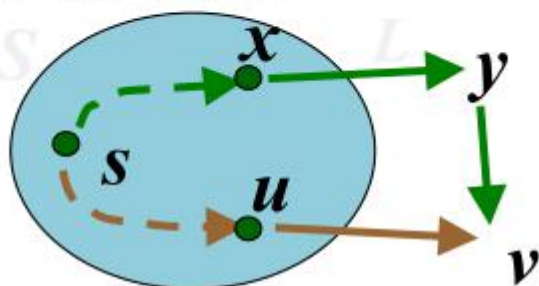
4.4 贪心法的典型应用

归纳步骤证明

假设命题对 k 为真, 考虑 $k+1$ 步算法
选择顶点 v (边 $\langle u, v \rangle$). 需要证明

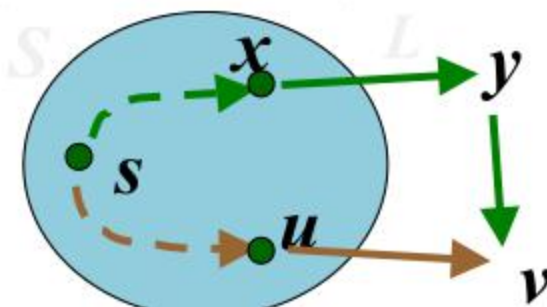
$$\text{dist}[v] = \text{short}[v]$$

若存在另一条 s - v 路径 L (绿色), 最后一次出 S 的顶点为 x , 经过 $V-S$ 的第一个顶点 y , 再由 y 经过一段在 $V-S$ 中的路径到达 v .



4.4 贪心法的典型应用

归纳步骤证明（续）



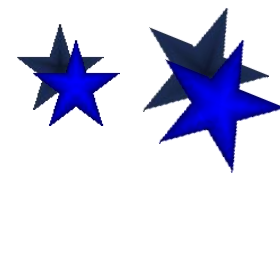
在 $k+1$ 步算法选择顶点 v ，而不是 y ，

$$\text{dist}[v] \leq \text{dist}[y]$$

令 y 到 v 的路径长度为 $d(y, v)$

$$\text{dist}[y] + d(y, v) \leq L$$

于是 $\text{dist}[v] \leq L$ ，即 $\text{dist}[v] = \text{short}[v]$



4.4 贪心法的典型应用

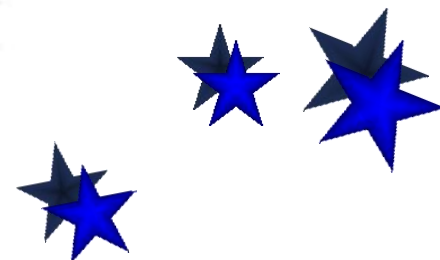
时间复杂度

- 时间复杂度: $O(nm)$

算法进行 $n-1$ 步

每步挑选1个具有最小 $dist$ 函数值的
结点进入 S , 需要 $O(m)$ 时间

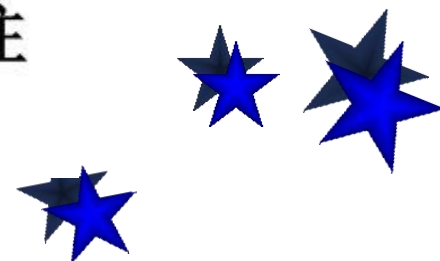
- 选用基于堆实现的优先队列的数据结构, 可以将算法时间复杂度降到 $O(m\log n)$



4.4 贪心法的典型应用

贪心法小结

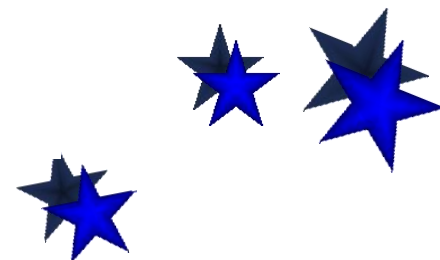
- 贪心法适用于组合优化问题.
- 求解过程是多步判断过程, 最终的判断序列对应于问题的最优解.
- 判断依据某种“短视的”贪心选择性质, 性质的好坏决定了算法的正确性. 贪心性质的选择往往依赖于直觉或者经验.



4.4 贪心法的典型应用

贪心法小结（续）

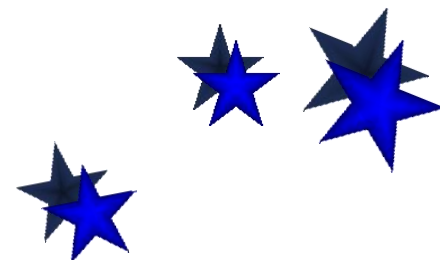
- 贪心法正确性证明方法：
 - (1) 直接计算优化函数，贪心法的解恰好取得最优值
 - (2) 数学归纳法（对算法步数或者问题规模归纳）
 - (3) 交换论证
- 证明贪心策略不对：举反例



4.4 贪心法的典型应用

贪心法小结（续）

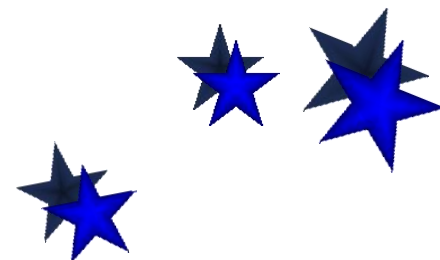
- 对于某些不能保证对所有的实例都得到最优解的贪心算法（近似算法），可做参数化分析或者误差分析。
- 贪心法的优势：算法简单，时间和空间复杂性低



4.4 贪心法的典型应用

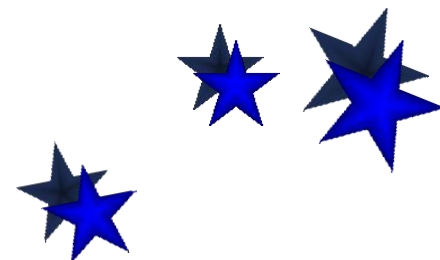
贪心法小结（续）

- 几个著名的贪心算法
最小生成树的Prim算法
最小生成树的Kruskal算法
单源最短路的Dijkstra算法



第五章 回溯与分支限界

- 回溯法：是一个类似穷举的搜索尝试过程，主要是在搜索尝试过程中寻找问题的解，当发现已经不能满足求解条件时就“回溯”（即回退），尝试其他路径，所以回溯法有“通用的解题法”之称。
- 使用回溯法求解的问题有两种
 - （1）求一个（或全部）可行解
 - （2）求最优解



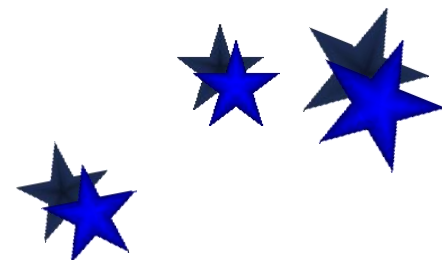
5.1 回溯算法的基本思想和适用条件

■ 几个回溯算法的例子

【例题1】 农夫（人）过河问题

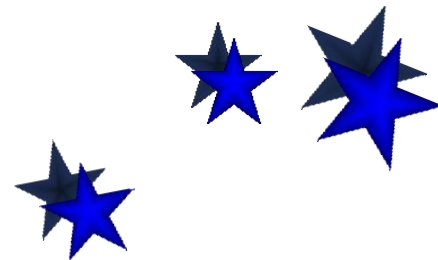
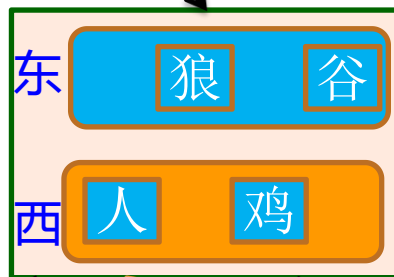
在河东岸有一个农夫、一只狼、一只鸡和一袋谷子，只有当农夫在现场时，狼不会把鸡吃掉，鸡也不会吃谷子，否则会出现这样的情况。

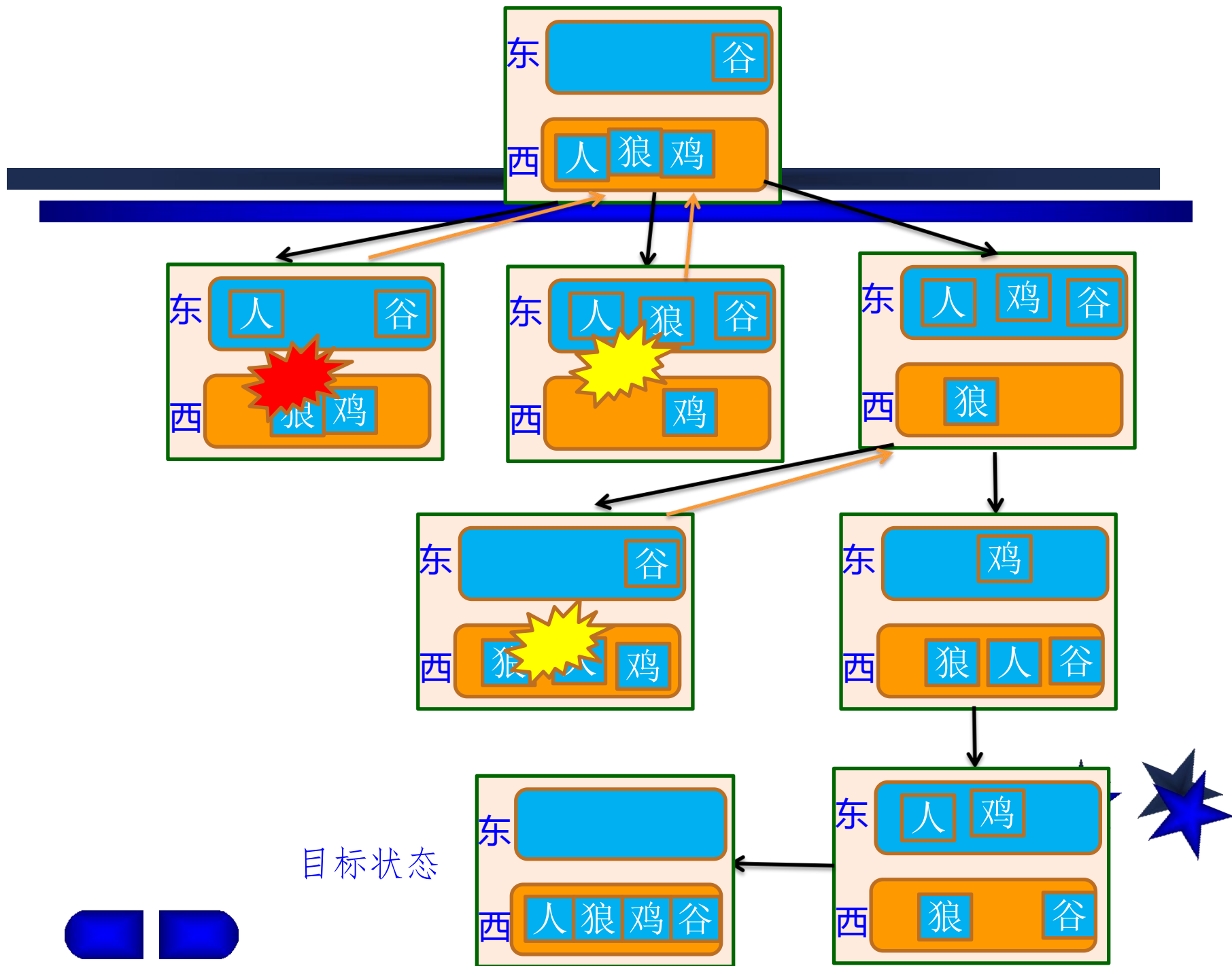
另有一条小船，该船只能由农夫操作，且最多只能载下农夫和另一样东西。设计一种过河方案，将农夫、狼、鸡和谷子借助小船运到河西岸。





初始状态





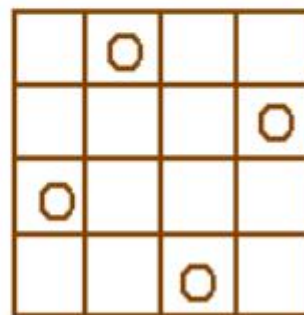
5.1 回溯算法的基本思想和适用条件

4后问题

4后问题：在 4×4 的方格棋盘上放置4个皇后，使得没有两个皇后在同一行、同一列、也不在同一条45度的斜线上。问有多少种可能的布局？

解是 4 维向量 $\langle x_1, x_2, x_3, x_4 \rangle$

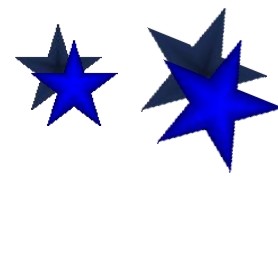
解： $\langle 2, 4, 1, 3 \rangle$, $\langle 3, 1, 4, 2 \rangle$



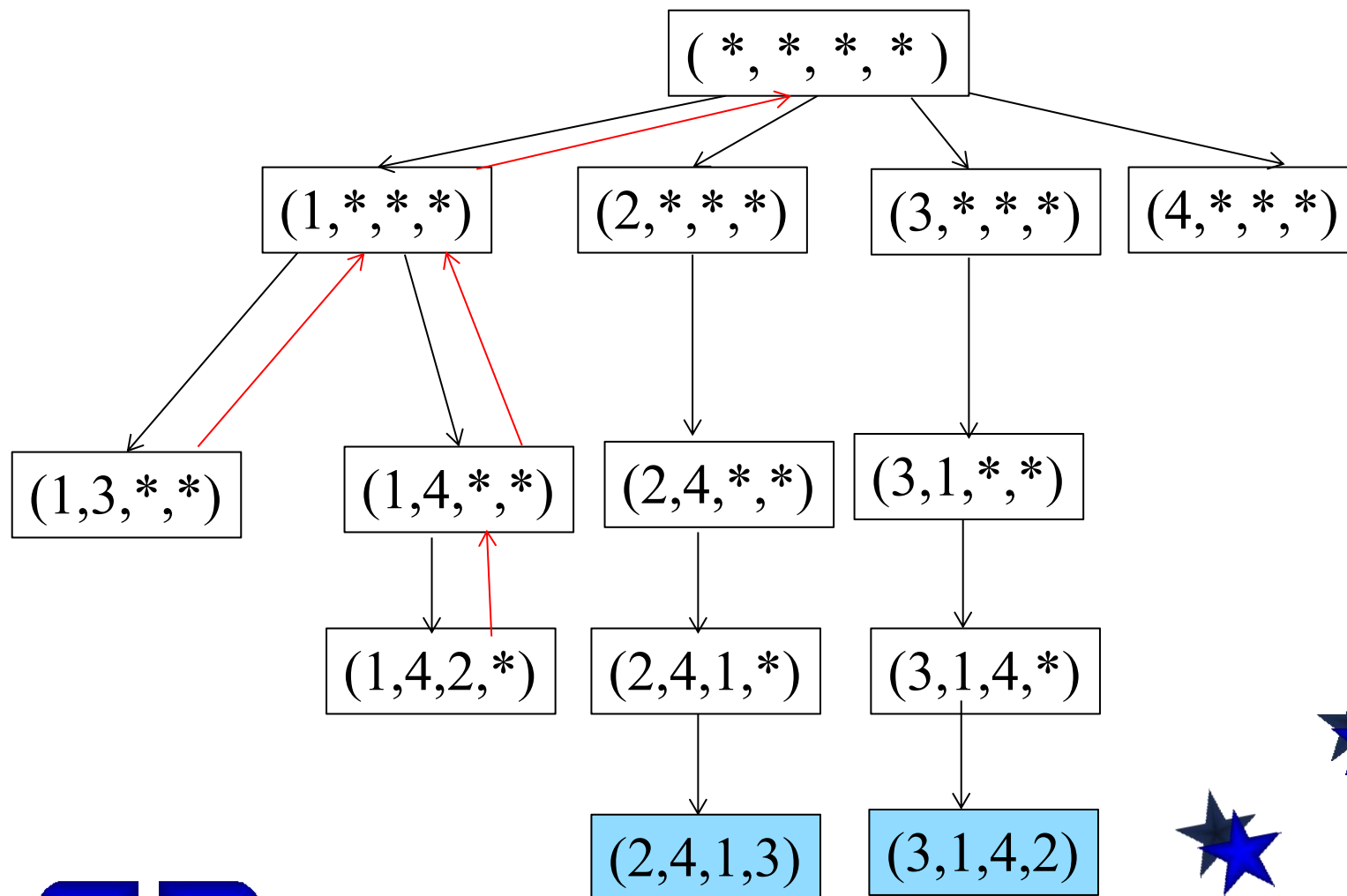
推广到8后问题

解： 8维向量，有 92个。

例如： $\langle 1, 5, 8, 6, 3, 7, 2, 4 \rangle$ 是解。

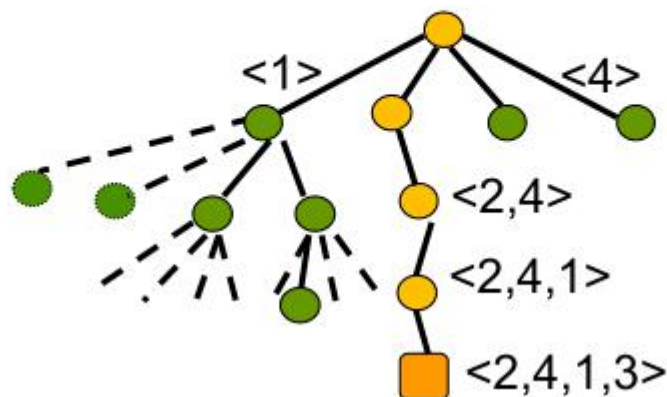


5.1 回溯算法的基本思想和适用条件



5.1 回溯算法的基本思想和适用条件

搜索空间：4叉树

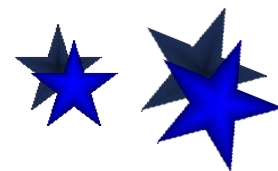


每个结点有4个儿子，分别代表选择
1,2,3,4列位置

第 i 层选择解向量中第 i 个分量的值

最深层的树叶是解

按深度优先次序遍历树，找到所有解



5.1 回溯算法的基本思想和适用条件

0-1背包问题

问题:

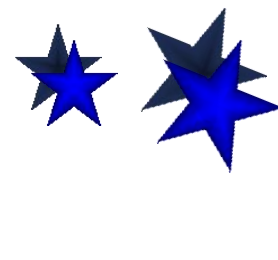
有 n 种物品, 每种物品只有 1 个. 第 i 种物品价值为 v_i , 重量为 w_i , $i=1,2,\dots,n$. 问如何选择放入背包的物品, 使得总重量不超过 B , 而价值达到最大?

实例:

$$V=\{12,11,9,8\}, W=\{8,6,4,3\}, B=13$$

最优解:

$$\langle 0,1,1,1 \rangle, \text{ 价值: } 28, \text{ 重量: } 13$$



5.1 回溯算法的基本思想和适用条件

算法设计

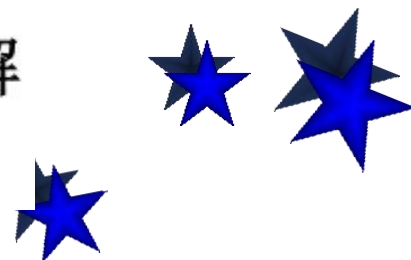
解： n 维0-1向量 $\langle x_1, x_2, \dots, x_n \rangle$,
 $x_i=1 \Leftrightarrow$ 物品 i 选入背包

结点： $\langle x_1, x_2, \dots, x_k \rangle$ （部分向量）

搜索空间： 0-1取值的二叉树，称为子集树，有 2^n 片树叶。

可行解：满足约束条件 $\sum_{i=1}^n w_i x_i \leq B$ 的解

最优解：可行解中价值达到最大的解



5.1 回溯算法的基本思想和适用条件

实例

输入:

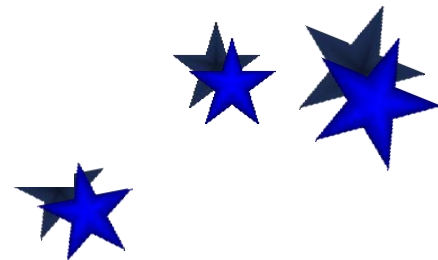
$V=\{12,11,9,8\}$, $W=\{8,6,4,3\}$, $B=13$

2个可行解:

$\langle 0,1,1,1 \rangle$, 选入物品2,3,4, 价值为28,
重量为13

$\langle 1,0,1,0 \rangle$, 选入物品1,3, 价值为21,
重量为12

最优解: $\langle 0,1,1,1 \rangle$

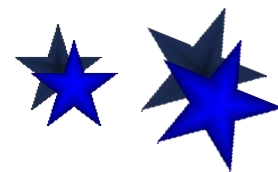
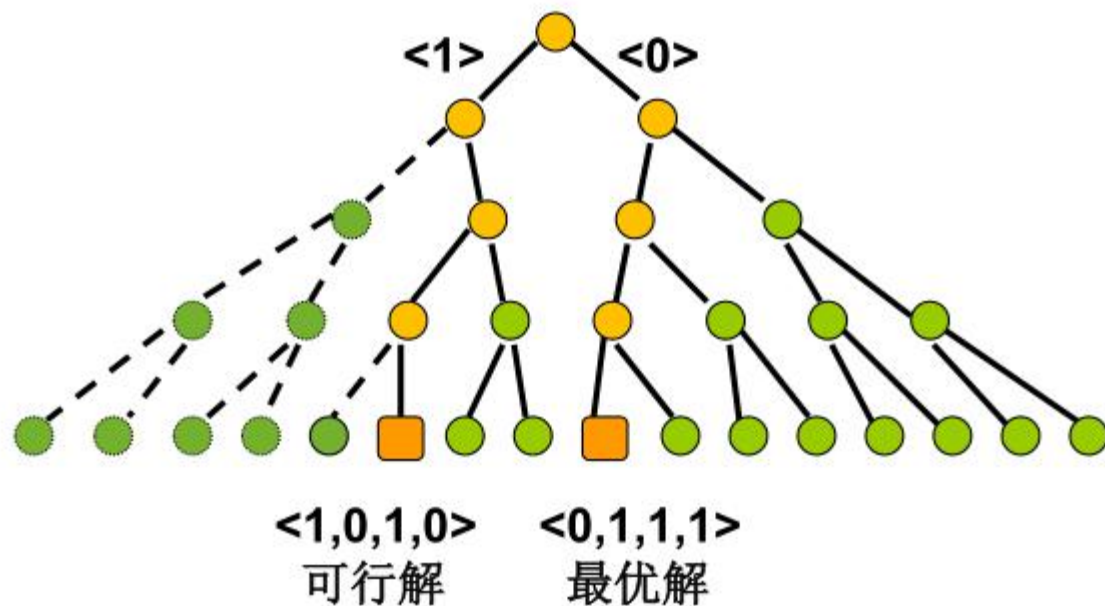


5.1 回溯算法的基本思想和适用条件

搜索空间

实例: $V=\{12,11,9,8\}$, $W=\{8,6,4,3\}$, $B=13$

搜索空间: 子集树, 2^n 片树叶



5.1 回溯算法的基本思想和适用条件

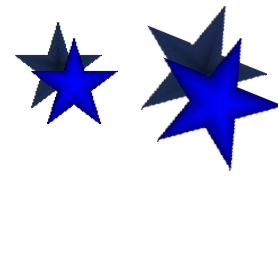
货郎问题

问题：有 n 个城市，已知任两个城市之间的距离，求一条每个城市恰好经过一次的回路，使得总长度最小。

建模：城市集 $C=\{c_1, c_2, \dots, c_n\}$ ，距离
 $d(c_i, c_j)=d(c_j, c_i) \in \mathbb{Z}^+, 1 \leq i < j \leq n$

求： $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$



5.1 回溯算法的基本思想和适用条件

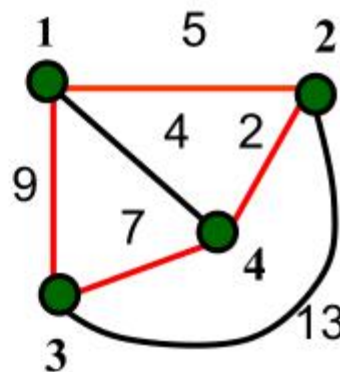
实例

$$C = \{1, 2, 3, 4\}$$

$$d(1, 2) = 5, d(1, 3) = 9,$$

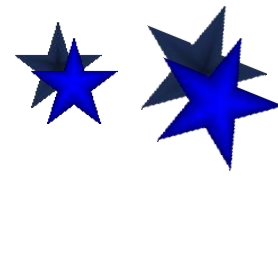
$$d(1, 4) = 4, d(2, 3) = 13,$$

$$d(2, 4) = 2, d(3, 4) = 7$$



解: $\langle 1, 2, 4, 3 \rangle$,

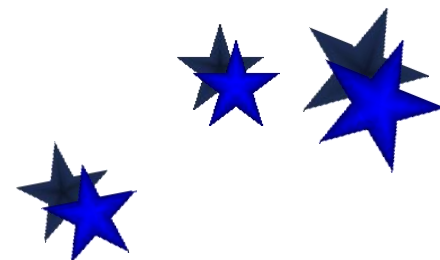
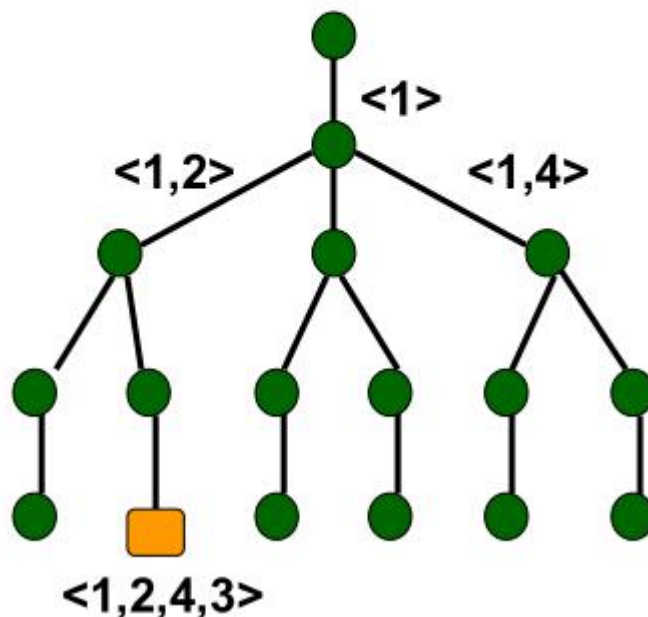
$$\text{长度} = 5 + 2 + 7 + 9 = 23$$



Government	Percentage
Current government	85%
Previous government	15%

搜索空间

排列树, 有 $(n-1)!$ 片树叶



5.1 回溯算法的基本思想和适用条件

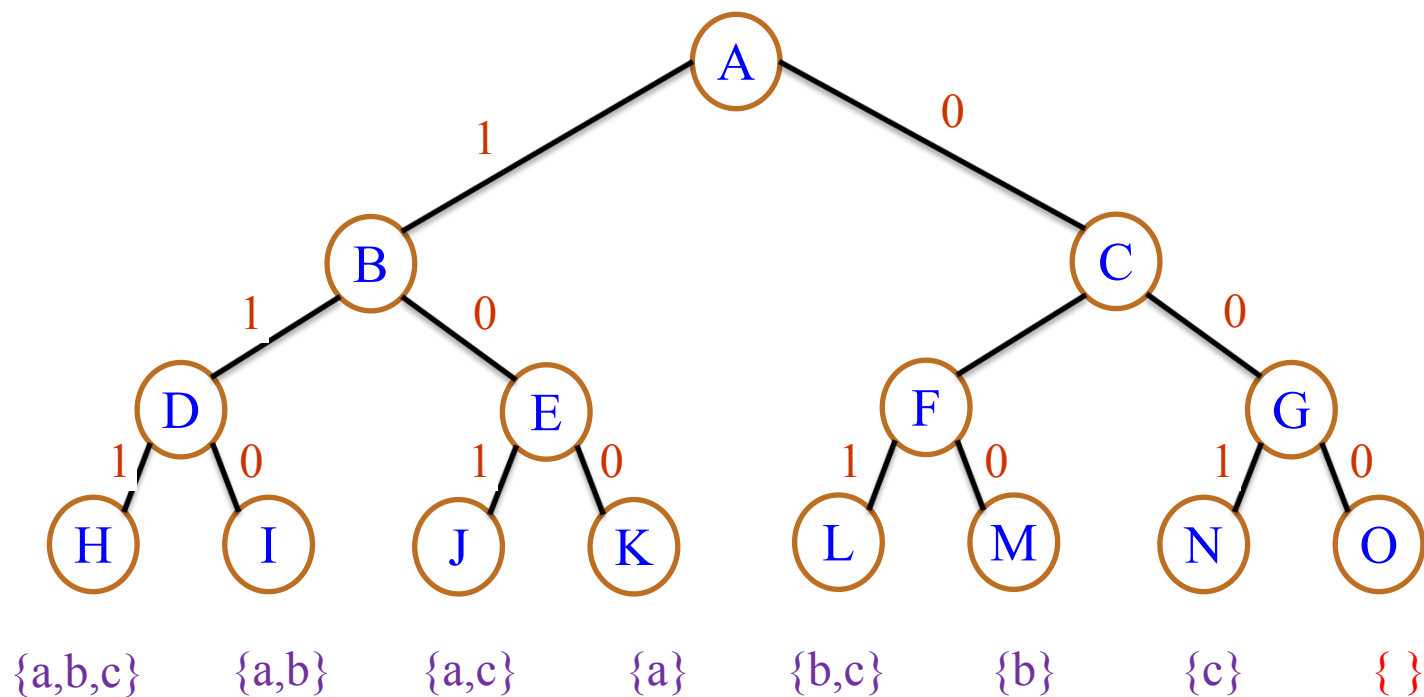
■ 问题的解空间

- 一个复杂问题的解决方案是由若干个小的决策步骤组成的决策序列。
- 一个问题的解可以表示成解向量： $X = (x_1, x_2, \dots, x_n)$
- 其中分量 x_i 对应第 i 步的选择，通常有两个或多个取值，表示为 $x_i \in S_i$, S_i 是 x_i 的取值候选集合。
- X 中各分量 x_i 所有取值集合构成问题的解向量空间，即解决一个问题的所有可能的决策序列构成该问题的解空间。
- 问题的解空间一般用树形式来组织，也称为解空间树或状态空间。

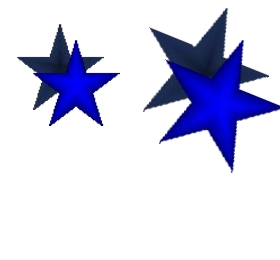
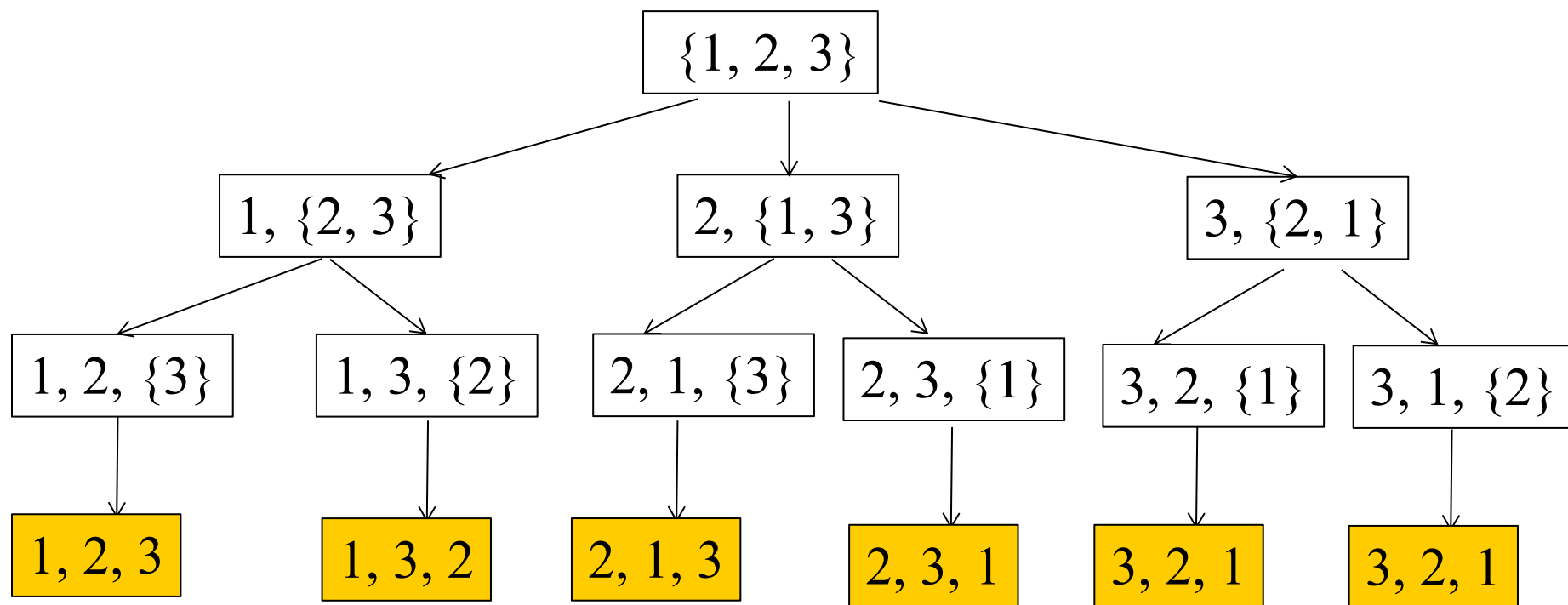


解空间树通常有两种类型：

■ **子集树**：当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的子集时，相应的解空间树称为子集树。



- **排列树**：当所给的问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。



5.1 回溯算法的基本思想和适用条件

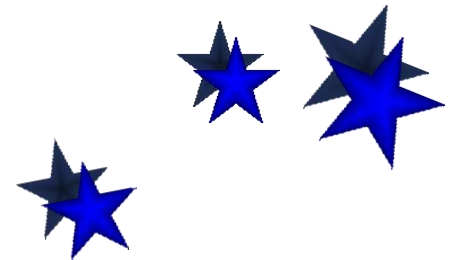
- 问题的解空间树是虚拟的，并不需要在算法运行时构造一棵真正的树结构，然后在该树中搜索问题的解，而是只存储从根结点到当前结点的路径。
- 实际上，有些问题的解空间因过于复杂或状态过多难以画出来。
- 树中的每一个结点确定所求解问题的一个问题状态。例如：树的根结点位于第1层，表示搜索的初始状态，第2层的结点表示对解向量的第一个分量做出选择后到达的状态，以此类推……，直到第 $n+1$ 层叶子结点。



5.1 回溯算法的基本思想和适用条件

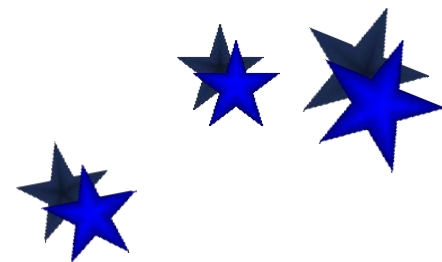
■ 什么是回溯法

- 在包含问题的所有解的解空间树中，按照**深度优先搜索**的策略，从根结点（开始结点）出发搜索解空间树。
- 活结点：指自身已生成但其孩子结点没有全部生成的结点。
- 扩展结点：扩展结点是指正在产生孩子结点的结点。
- 在当前的扩展结点处，搜索向纵深方向移至一个新结点。这个新结点就成为新的活结点，并成为当前扩展结点。



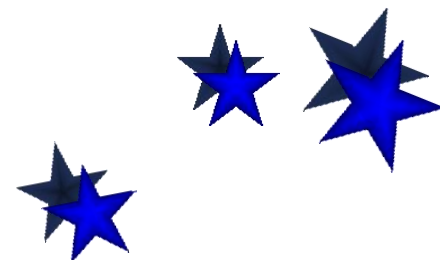
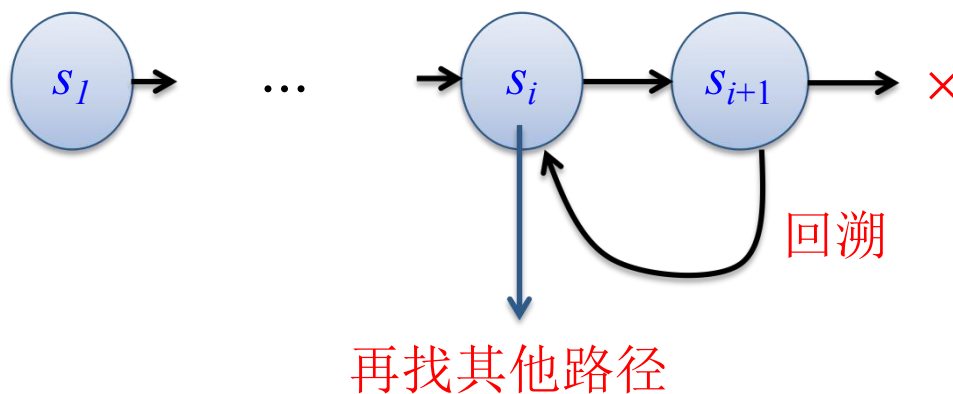
5.1 回溯算法的基本思想和适用条件

- 如果在当前的扩展结点处不能再向纵深方向移动，则当前扩展结点就成为死结点（死结点是指由根结点到该结点构成的部分解不满足约束条件，或者其子结点已经搜索完毕）
- 此时应往回移动（回溯）至最近的一个活结点处，并使这个活结点成为当前的扩展结点。
- 回溯法以这种方式递归地在解空间中搜索，直至找到所要求的解或解空间中已无活结点为止。



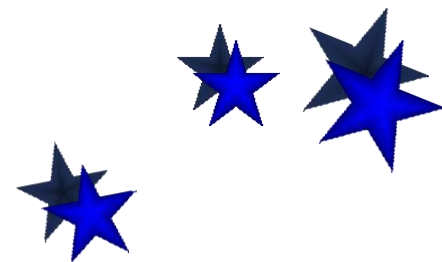
5.1 回溯算法的基本思想和适用条件

- 当从状态 s_i 搜索到状态 s_{i+1} 后，如果 s_{i+1} 变为死结点，则从状态 s_{i+1} 回退到 s_i ，再从 s_i 找其他可能的路径，所以回溯法体现出走不通就退回再走的思路。



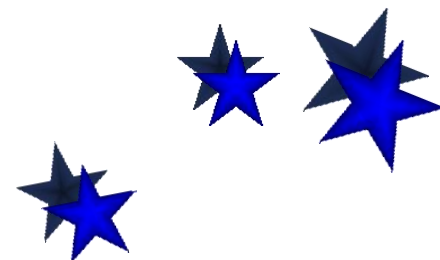
5.1 回溯算法的基本思想和适用条件

- 在状态空间树中，每个叶子结点对应一个求解状态，每个非叶子结点对应一个部分解状态。
- 解空间树是很规范的，数组 a 的元素个数为 n ，则解空间树的高度就为 $n+1$ 。第 i 层对应元素 a_i 的决策。
- 通常情况下，从根结点到叶子结点（不含搜索失败的结点）的路径构成了解空间的一个可行解。
- 求幂集问题，属于求全部可行解问题，因此问题的解就是全部解空间；若问题修改为求子集最大和问题，就是一个求最优解问题，是解空间的一个子集。



5.1 回溯算法的基本思想和适用条件

- 若用回溯法求问题的所有解时，需要回溯到根结点，且根结点的所有可行的子树都要已被搜索完才结束。而若使用回溯法求任一个解时，只要搜索到问题的一个解就可以结束。
- 由于采用回溯法求解时存在退回到祖先结点的过程，所以需要保存搜索过的结点，通常有两种方法：
 - (1) 使用栈保存祖先结点。
 - (2) 使用递归方法。

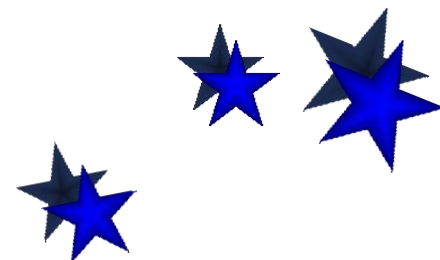


5.1 回溯算法的基本思想和适用条件

- 回溯法搜索解空间时，通常采用两种策略避免无效搜索，提高回溯的搜索效率：

- 用约束函数在扩展结点处剪除不满足约束的子树；
- 用限界函数剪去得不到问题解或最优解的子树。

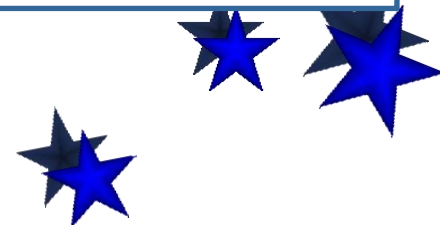
这两类函数统称为剪枝函数。



5.1 回溯算法的基本思想和适用条件

归纳起来，用回溯法解题的一般步骤如下：

- ① 针对所给问题，确定问题的解空间树，问题的解空间树应至少包含问题的一个（最优）解。
- ② 确定结点的扩展搜索规则。
- ③ 以深度优先方式搜索解空间树，并在搜索过程中可以采用剪枝函数来避免无效搜索。



5.1 回溯算法的基本思想和适用条件

回溯算法的适用条件

在结点 $\langle x_1, x_2, \dots, x_k \rangle$ 处

$P(x_1, x_2, \dots, x_k)$ 为真

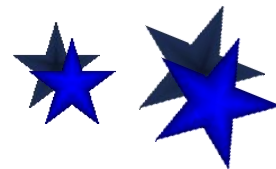
$\Leftrightarrow$ 向量 $\langle x_1, x_2, \dots, x_k \rangle$ 满足某个性质
(n 后中 k 个皇后放在彼此不攻击的位置)

多米诺性质

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

$$\neg P(x_1, x_2, \dots, x_k) \rightarrow \neg P(x_1, x_2, \dots, x_{k+1}) \quad 0 < k < n$$

k 维向量不满足约束条件, 扩张向量到
 $k+1$ 维仍旧不满足, 可以回溯.



5.1 回溯算法的基本思想和适用条件

一个反例

例 求不等式的整数解

$$5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_k \leq 3, \quad k=1,2,3$$

$P(x_1, \dots, x_k)$: 将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分, 部分和小于等于10

不满足多米诺性质:

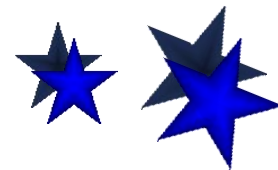
$$5x_1 + 4x_2 - x_3 \leq 10 \not\Rightarrow 5x_1 + 4x_2 \leq 10$$

变换使得问题满足多米诺性质:

令 $x_3 = 3 - x_3'$,

$$5x_1 + 4x_2 + x_3' \leq 13$$

$$1 \leq x_1, \quad x_2 \leq 3, \quad 0 \leq x_3' \leq 2$$



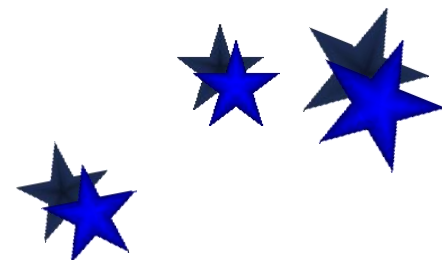
5.1 回溯算法的基本思想和适用条件

■ 回溯法与深度优先遍历的异同

- 两者的相同点：回溯法在实现上也是遵循深度优先的，即一步一步往前探索。

【举例】假设图G使用邻接表存储，设计一个算法输出图G中从顶点u到v的**所有**简单路径。

(1) 若仅使用深度优先遍历，则只能找到一条u到v的路径：
因为每个访问过结点只访问一次($visit[u]=1$)



5.1 回溯算法的基本思想和适用条件

(2) 若使用回溯算法（深度遍历+回溯），才可以找到所有的u到v的路径，访问结点时 $visited[u]=1$ ，回溯到该结点时再将 $visited[u]=0$ 。

● 两者的不同点

(1) **访问次数的不同**：深度优先遍历对已经访问过的顶点不再访问，所有顶点仅访问一次。而回溯法中已经访问过的顶点可能再次访问（在回溯时将访问标识修改为1）。

(2) **剪枝的不同**：深度优先遍历不含剪枝，而很多回溯算法采用剪枝条件剪除不必要的分枝以提高效能。

